B SMIT | AI & DATA SCIENCE

AI Bug Report Analyzer

Built with LangGraph, Gemini and FastAPI

SUBMITTED BY

Anushay Ayat

BATCH

12

COURSE

AI and Data Science

SUBMISSION DATE

22 / 10 / 2026

Agenda

What this presentation covers

01 Problem & objective

02 Solution overview

03 LangGraph workflow

04 System architecture

05 Backend implementation

06 Frontend & user experience

07 Screenshots / demo

08 Testing & challenges

09 Future scope & conclusion

Problem & Objective

Why a bug report analyzer?

The problem

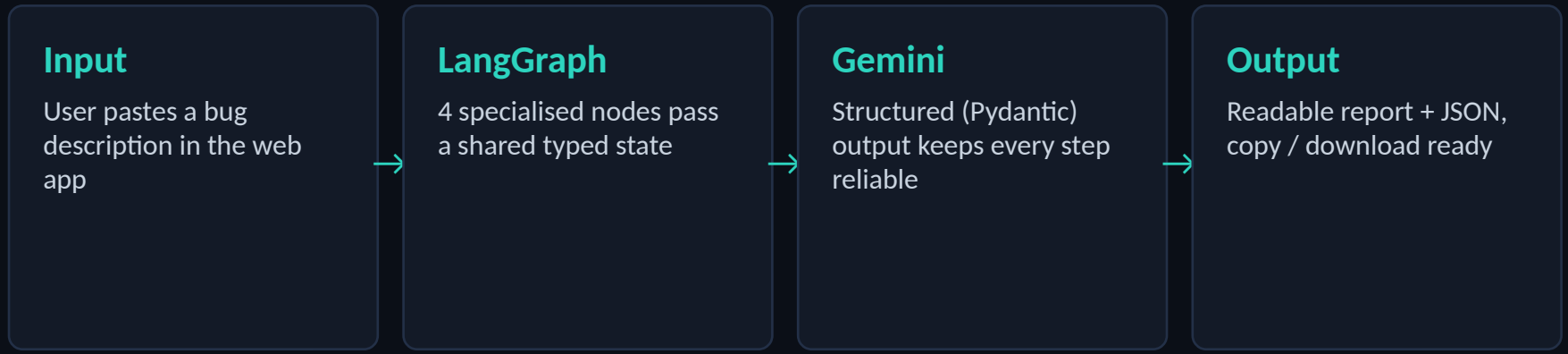
- Bug reports are written in free-form, unstructured language
- Developers waste time asking for missing details
- Severity is judged inconsistently between reporters
- Root-cause hunting starts from zero every time

The objective

- Accept a bug description in natural language
- Extract title, feature and summary automatically
- Classify severity: Critical, High, Medium, Low
- Suggest likely root causes
- Produce one clean, structured bug report

Solution Overview

A multi-step AI workflow instead of a single prompt



Why LangGraph? Each task is isolated, testable and easy to extend - e.g. adding a duplicate-detection node later needs only one new node and one edge.

LangGraph Workflow

START → 4 nodes → END



Shared state: bug_description → title, affected_feature, summary → severity, severity_reason → root_causes → steps, next_steps, final_report

Technology Stack

Modern, lightweight and easy to deploy

Backend

Python 3.10+, FastAPI, Uvicorn

AI orchestration

LangGraph (StateGraph), LangChain

LLM

Google Gemini via langchain-google-genai

Validation

Pydantic v2 structured output

Frontend

HTML5, CSS3, vanilla JavaScript

Streaming

Server-Sent Events (SSE)

Project Structure

Clean separation of concerns

```
ai-bug-report-analyzer/
├── backend/app/
│   ├── main.py        FastAPI entry
│   ├── config.py      settings (.env)
│   ├── schemas.py     Pydantic models
│   ├── api/routes.py  REST + SSE
│   ├── graph/
│   │   ├── state.py   BugState
│   │   ├── prompts.py prompts
│   │   ├── nodes.py   4 nodes
│   │   └── workflow.py graph
│   └── services/  llm + demo
├── frontend/  (html, css, js)
├── tests/    docs/    README.md
```

Design principles

- Prompts separated from logic
- One file per responsibility
- LLM access isolated in services/
- Graph wiring in one place
- Frontend served by FastAPI
- Config via environment variables

State & Node Implementation

Each node returns a partial state update

```
class BugState(TypedDict, total=False):
    bug_description: str
    title: str
    affected_feature: str
    summary: str
    severity: str
    severity_reason: str
    root_causes: List[str]
    steps_to_reproduce: List[str]
    suggested_next_steps: List[str]
    final_report: str

g.add_edge(START, "bug_analysis")
g.add_edge("bug_analysis",
           "severity_detection")
...  g.add_edge("format_report", END)
```

How it works

- Node calls Gemini with a focused prompt
- with_structured_output(Schema) returns validated Pydantic objects
- Result is merged into shared state
- Format node renders the final Markdown report
- Any LLM failure falls back to the offline engine

Severity Classification

Four levels with clear definitions

Critical	Crash, data loss, security hole or payments fully blocked
High	Major feature broken, no reasonable workaround
Medium	Feature degraded or inconsistent, workaround exists
Low	Cosmetic issue or minor inconvenience

REST API

Interactive docs available at /docs

Method	Endpoint	Purpose
GET	/api/health	Status, version and active engine
GET	/api/samples	Sample bug descriptions for the UI
POST	/api/analyze	Run workflow, return full JSON report
POST	/api/analyze/stream	Stream one SSE event per finished node

```
Request body:  { "description": "Users cannot log in after the latest update..." }   (10 - 4000 characters, validated by Pydantic)
```

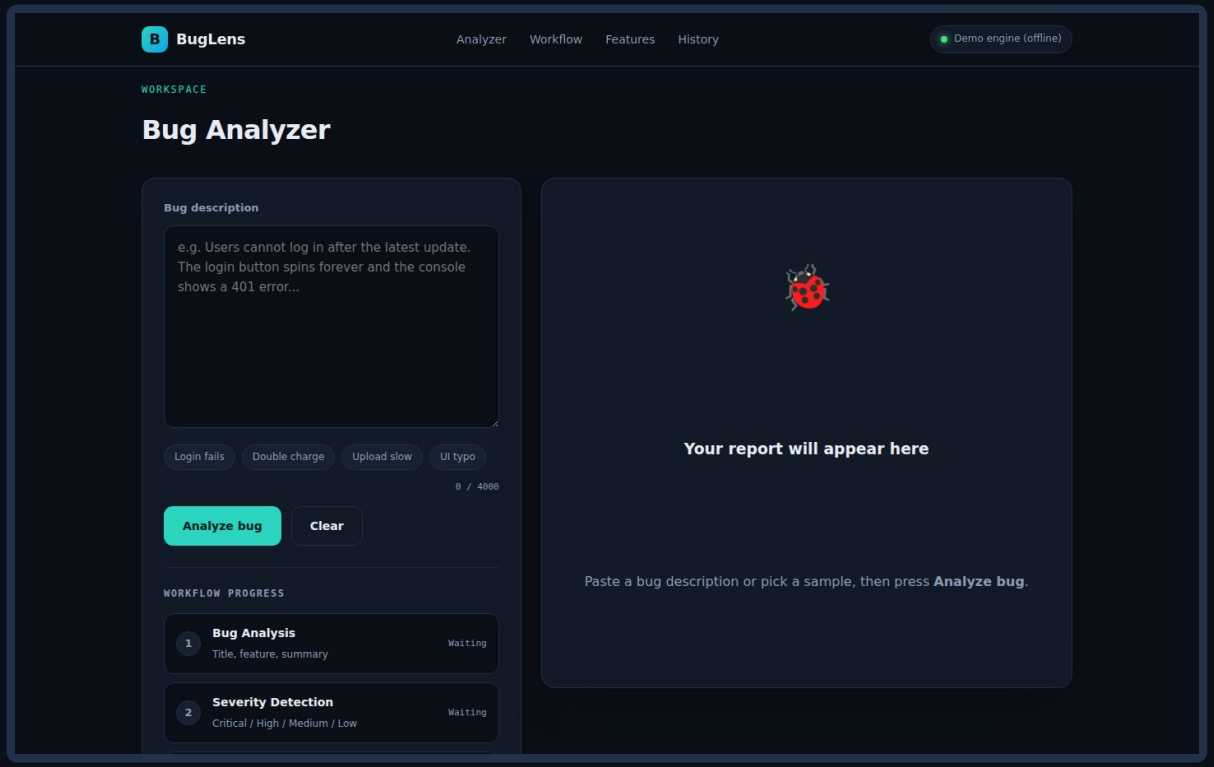
Home page

Hero section with clear call to action



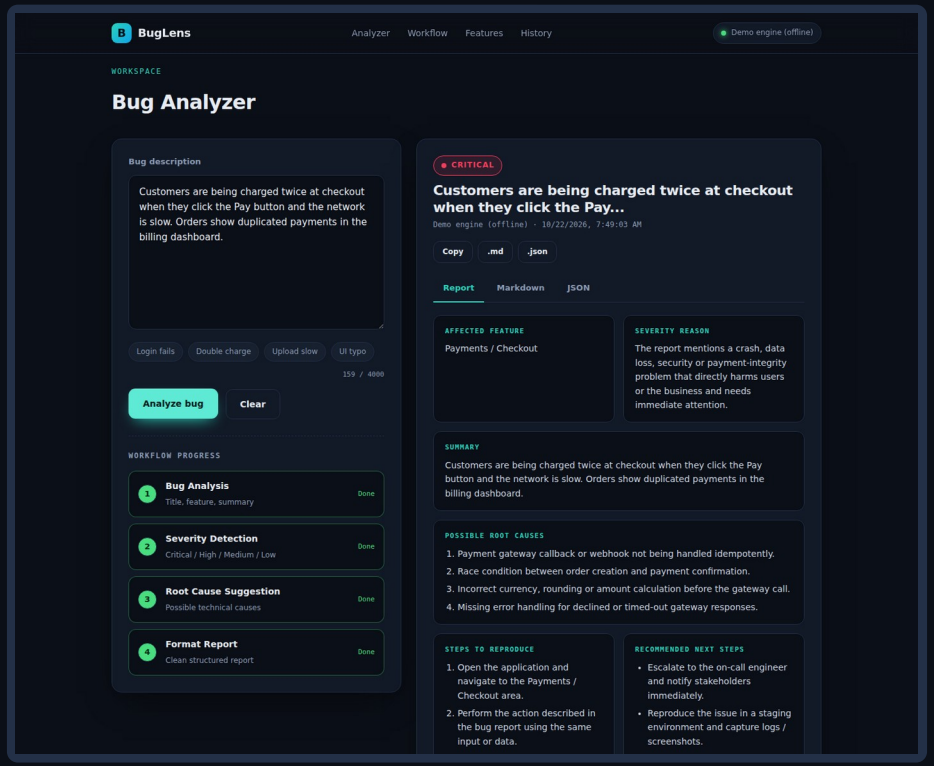
Bug analyzer workspace

Input, sample bugs and workflow tracker



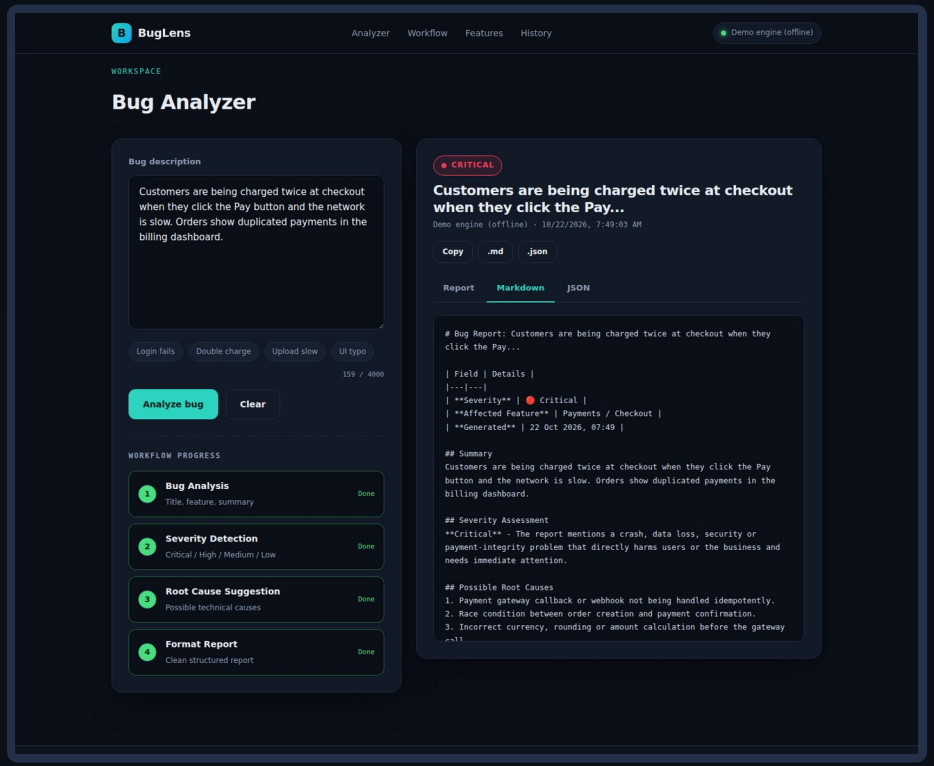
Live workflow & report

Nodes finish one by one; report appears on the right



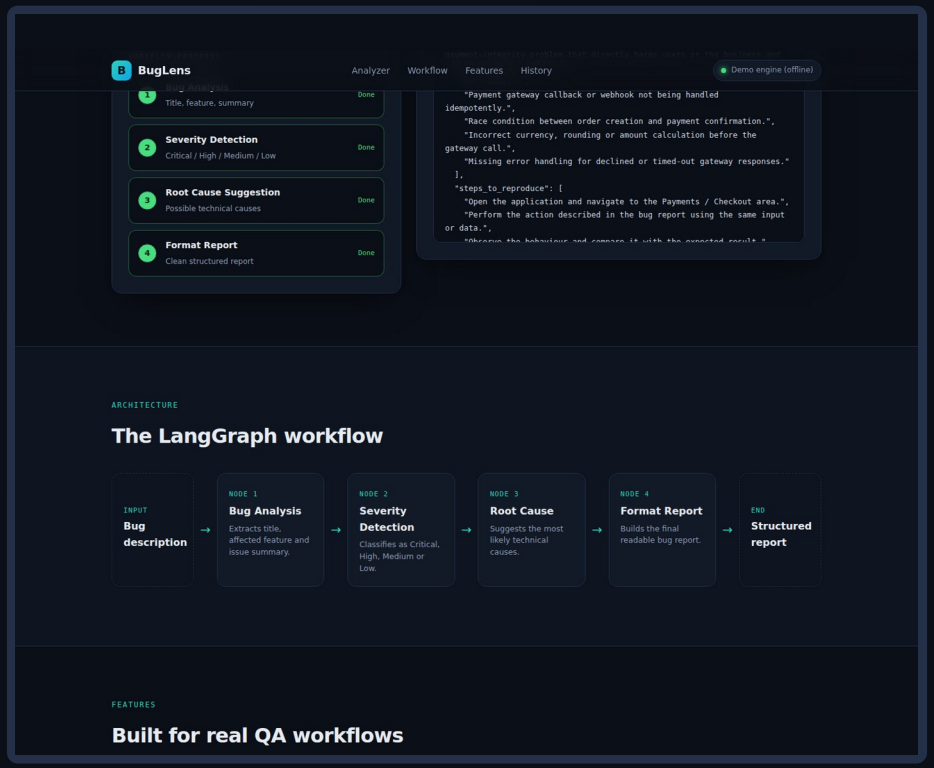
Markdown & JSON output

Developer-ready exports



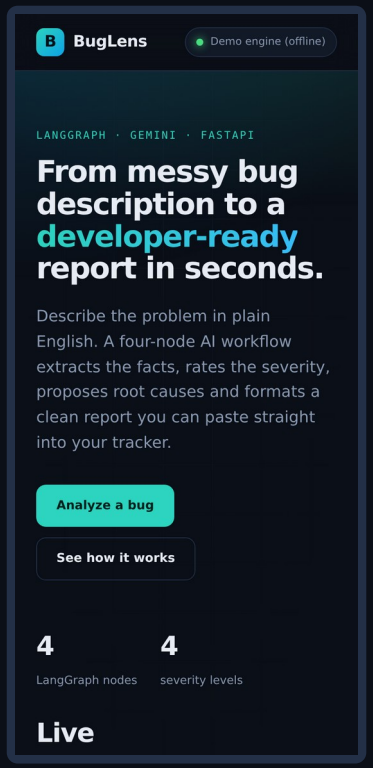
Architecture & features

Workflow explainer section



Responsive design

Works on phones as well as desktops



Frontend highlights

- Dark, professional design system with CSS variables
- Fully responsive grid (desktop, tablet, mobile)
- Real-time stepper driven by Server-Sent Events
- Severity badges: red, orange, yellow, green
- Copy, .md and .json export buttons
- Local history of the last six analyses

Sample Output

Input → structured bug report

Input

Customers are being charged twice at checkout when they click the Pay button and the network is slow. Orders show duplicated payments in the billing dashboard.

→

Output (excerpt)

Severity: Critical
Feature: Payments / Checkout
Root causes:
1. Gateway callback not handled idempotently
2. Race condition between order creation and payment confirmation
Next steps: escalate to on-call, reproduce on staging, add regression test

Testing & Quality

Automated tests with pytest

What is tested

- Full graph run returns every state field
- Critical vs Low severity classification
- /api/health and /api/analyze endpoints
- Input validation (too-short text → 422)

Reliability measures

- Pydantic validates all LLM output
- Fallback engine if Gemini fails or key missing
- Input length limits on the API
- Clear error messages shown in the UI

Challenges & Solutions

Unreliable free-text LLM output	→	Used structured output with Pydantic schemas for every node
App unusable without API key	→	Built an offline rule-based fallback engine
Users wait without feedback	→	Streamed node-by-node progress with SSE
Keeping nodes independent	→	Typed shared state + partial updates per node

Future Enhancements

Where the project can grow

- Duplicate bug detection with embeddings
- Direct export to Jira / GitHub Issues
- Screenshot and log-file analysis (multimodal)
- User accounts and a persistent database
- Team dashboards with severity analytics
- Docker deployment and CI pipeline

Conclusion

What was achieved

- Implemented the complete 4-node LangGraph workflow from the assignment
- Delivered a professional full-stack application: FastAPI backend + responsive website
- Structured Gemini output, live progress, exports and history go beyond the brief
- Clean folder structure, documentation, tests and screenshots included
- Ready to extend with more nodes and integrations

Thank You

Questions?

Anushay Ayat | AI and Data Science | Batch 12 | SMIT